

CIVILEG2 - DEVELOPMENT PLAN ANALYSIS

تحليل شامل لخطة التطوير الخمسية

ملاحظات تفصيلية وتوصيات فنية لكل مرحلة من مراحل خطة تطوير تطبيق
المهندس المدني

يوليو 2026 | الإصدار 1.1.0

0

لم تبدأ

4

مراحل جزئية

1

مرحلة مكتملة

المرحلة الأولى

تحسين سرعة بناء Gradle

مكتمل 100%

تم تنفيذ هذه المرحلة بالكامل في التزام الأخير (5137e74). شملت التحسينات رفع ذاكرة JVM من 2 جيجابايت إلى 4 جيجابايت لتسريع معالجة التعليقات التوضيحية (KSP/Hilt)، وتفعيل خيار التخزين المؤقت للتكوين (configuration-cache) الذي يسرع عمليات إعادة البناء بشكل ملحوظ، بالإضافة إلى استخدام مترجم Kotlin في نفس العملية (in-process) بدلا من عملية منفصلة. كما تم تعطيل الميزات غير المستخدمة مثل AIDL Shadersg RenderScriptg لتقليل وقت المعالجة.

ما تم تنفيذه فعلا

زيادة كومة JVM إلى 4 جيجابايت - كومة Kotlin Daemon إلى 2 جيجابايت - تفعيل التخزين المؤقت للتكوين - استخدام مترجم Kotlin في نفس العملية - تعطيل AIDL Shadersg RenderScriptg - تفعيل البناء المتوازي والتخزين المؤقت

ملاحظة إضافية مقترحة

يمكن إضافة ميزة Build Scan عبر Gradle Enterprise لتحليل أداء البناء بشكل أعمق، وكذلك تفعيل Kotlin Incremental Compilation بشكل صريح في إعدادات Kotlin. يوجد أيضا إمكانية تفعيل Gradle Build Cache عن بُعد لفريق العمل إذا تم التوسع مستقبلا.

المرحلة الثانية

الهندسة المعمارية (MVVM / التنظيم النمطي)

مكتمل 40% - يحتاج عمل جوهري

40%

نسبة الإنجاز

المشروع يمتلك أساسا جيدا في نمط MVVM مع 16 ViewModel وحقق التبعية عبر Hilt ونمط الاستراتيجية (Strategy Pattern) في محرك الحسابات. لكن هناك مشاكل هيكلية جوهرية تحتاج إلى معالجة عاجلة قبل التوسع في الميزات الجديدة. أبرز هذه المشاكل وجود نظام حسابات مزدوج (CalculatorEngine القديم مقابل التسلسل الهرمي الجديد في domain/calculations)، وعدم اكتمال نمط Repository لمعظم الميزات، وغياب UseCases باستثناء اثنين فقط.

المشكلة الأولى: نظام الحسابات المزدوج

خطأ جوهري - CalculatorEngine (82 كيلوبايت) مقابل domain/calculations

يوجد في المشروع نظامان متوازيان للحسابات: الأول هو CalculatorEngine في حزمة utils وهو ملف عملاق يضم جميع الحسابات في فئة واحدة (God Class). الثاني هو التسلسل الهرمي المنظم في domain/calculations الذي يستخدم نمط الاستراتيجية مع فئات أساسية مجردة وتطبيقات منفصلة لكل كود تصميم (ECP/ACI/SBC). هذا الازدواج يخلق ارتباكاً في تحديد أي نظام هو المصدر الصحيح للحسابات، ويضعف جهود الصيانة والتطوير.

التوصية الفنية هي إزالة CalculatorEngine تدريجياً ونقل أي منطق حسابي فريد غير موجود في domain/calculations إلى الفئات المنظمة. يجب أن يكون domain/calculations هو المصدر الوحيد للحسابات، مع التأكد من أن جميع ViewModels تستخدم CalculationFactory للحصول على تطبيقات الكود المناسبة بدلا من الاستدعاء المباشر لـ CalculatorEngine.

المشكلة الثانية: عدم اكتمال نمط Repository

فقط Project يمتلك Repository منفصل (واجهة + تطبيق)

من بين 11 كيانا في قاعدة البيانات، فقط Project يمتلك فصل طبقة Repository بشكل صحيح مع واجهة في domain/repository وتطبيق في data/repository. باقي الكيانات (Beam, Column, Slab, Footing, Stair, RetainingWall, Tank, Inventory) يتم الوصول إلى قواعد بياناتها مباشرة من ViewModels أو عبر DesignRepository الذي يقع في حزمة db بدلا من data/repository مما ينتهك فصل الطبقات.

يجب إنشاء واجهات Repository لكل كيانات وتطبيقاتها في data/repository، مع نقل DesignRepository من حزمة db إلى حزمة data/repository لتوحيد البنية. كما يجب إنشاء UseCases لكل عملية حسابية رئيسية (تصميم العنصر، التحقق من المدخلات، حساب BOQ) بدلا من ترك المنطق في ViewModels.

المشكلة الثالثة: مشروع أحادي الوحدة

كل الكود في وحدة: app واحدة - بدون تنظيم نمطي

المشروع حاليا يتكون من وحدة واحدة فقط (app) تحتوي على كل شيء من واجهات المستخدم إلى محركات الحسابات إلى قاعدة البيانات. هذا يعني أن أي تغيير في مكون يتطلب إعادة بناء التطبيق بالكامل، كما أن الاختبارات المعزولة للوحدات الفردية تصبح صعبة التنفيذ. البنية الحالية (حزم domain/data/ui) توفر فصلا منطقيا لكنها غير مفروضة حدوديا بواسطة Gradle.

التنظيم النمطي المقترح يتضمن فصل المشروع إلى: وحدة: core (أنواع أساسية مشتركة وأدوات مساعدة)، وحدة: domain (كيانات وواجهات Repositories)، وحدة: data (تنفيذ Repositories وقاعدة البيانات DataStore)، وحدة: feature-design (شاشات التصميم الهيكلي)، وحدة: feature-tools (الأدوات المساعدة)، وحدة: app (نقطة الدخول Hilt والتجميع).

ملاحظات إضافية على الهيكل المعماري

المشكلة	الأثر	الأولوية
تكرار الكيانات (domain/entities مقابل db/)	تعقيد في التحويل بين الطبقات واحتمال عدم تناسق البيانات	عالية
DesignRepository في حزمة db/ بدلا من /data	انتهاك فصل الطبقات وصعوبة في الاختبار المعزول	عالية
وجود UseCase واحد فقط لكل من BOQ و Inventoryg	منطق العمل مبعثر في ViewModels بدلا من طبقة منفصلة	متوسطة
هجرة Compose غير مكتملة (~15 شاشة XML)	ازدواج في أنماط واجهة المستخدم وصعوبة الصيانة	متوسطة
استخدام fallbackToDestructiveMigration في Room	فقدان بيانات المستخدم عند تحديث إصدار قاعدة البيانات	عالية

المرحلة الثالثة

تكامل قاعدة البيانات و Firebase

مكتمل 35% - Room جاهزة، Firebase غير موجود

35%

نسبة الإنجاز

تم دمج Room بشكل كامل مع 11 كيانا و11 واجهة DAO ونظام تحويلات (Converters) يتعامل مع التواريخ وحالات المشروع وأنواع التصميم. قاعدة البيانات في الإصدار السادس مع دعم KSP للمعالجة التوضيحية.

لكن هناك مشكلة حرجية في استراتيجية الترحيل (Migration) حيث يستخدم المشروع fallbackToDestructiveMigration مما يعني أن أي تحديث لقاعدة البيانات سيؤدي إلى حذف جميع بيانات المستخدم. هذا غير مقبول لتطبيق إنتاجي.

حالة Room الحالية

البنية موجودة وتعمل

11 كيانا (Project, Design, MaterialItem, Footing, ColumnEntity, Slab, Beam, Stair, RetainingWall,) DAO واجهة 11 - فئة Converters للتاريخ والحالات - إصدار قاعدة البيانات 6 - معالجة عبر KSP بدلا من kapt

مشكلة حرجية: استراتيجية الترحيل المدمرة

استخدام fallbackToDestructiveMigration() يعني أن كل تغيير في schema قاعدة البيانات (إضافة عمود، تعديل نوع، إضافة جدول) سيحذف جميع بيانات المستخدم السابقة. يجب كتابة Migration لكل إصدار جديد. مع 11 جدولاً، يجب البدء فوراً في إنشاء ملفات Migration قبل أي إصدار إنتاجي.

Firebase - غير موجود بالكامل

لا يوجد أي تكامل مع Firebase

لا توجد أي تبعيات Firebase في المشروع. لا يوجد ملف google-services.json، ولا أي كلاس مرتبط بـ Firebase. الخدمات غير المتوفرة تشمل: Crashlytics لتتبع الأعطال، Analytics لتحليل استخدام التطبيق، Cloud Firestore للمزامنة السحابية، Remote Config للإعدادات عن بُعد، Authentication للمصادقة.

لتكامل Firebase المطلوب، يجب أولاً إضافة مكتبة Google Services في ملف build.gradle.kts للمشروع، ثم إضافة التبعيات المطلوبة. الأولوية القصوى هي Crashlytics لتتبع الأعطال في الإصدار الإنتاجي لأنه لا يمكن إصلاح مشاكل لا تعرف عنها. يليه Analytics لفهم سلوك المستخدمين وأي الشاشات الأكثر استخداماً. Cloud Firestore يأتي لاحقاً عند الحاجة إلى مزامنة البيانات بين الأجهزة أو النسخ الاحتياطي السحابي.

الخدمة	الغرض	الأولوية
Firebase Crashlytics	تتبع الأعطال والتقلصات في الإصدار الإنتاجي	درجة
Firebase Analytics	تحليل سلوك المستخدمين وأداء الشاشات	عالية
Room Migration Strategy	حماية بيانات المستخدم عند تحديث قاعدة البيانات	درجة
Cloud Firestore	مزامنة سحابية ونسخ احتياطي للمشاريع	متوسطة
Remote Config	تعديل الإعدادات عن بُعد بدون تحديث التطبيق	منخفضة
Firebase Authentication	تسجيل دخول المستخدمين وحسابات متعددة	منخفضة

المرحلة الرابعة

التحقيق المالي (الاشتراكات / الإعلانات)

مكتمل 30% - البنية موجودة لكنها بالهوية التجريبية

30%

نسبة الإنجاز

نظام الإعلانات مدمج بشكل جيد من الناحية الهيكلية مع 3 ملفات مخصصة (AdManager, AdComponents, AdPreferences) وفلسفة إعلانات غير مزعجة. يدعم النظام البانرات والإعلانات بينية والمكافآت والإعلانات الأصلية مع تحكم في التكرار (إعلان بيئي كل 3 عمليات تصميم). كما يدعم كتم الإعلانات للمستخدمين المميزين وموافقة GDPR عبر UMP. لكن جميع معرفات الإعلانات لا تزال معرفات اختبار من Google ويجب استبدالها قبل الإطلاق.

مشكلة: جميع معرفات AdMob هي معرفات اختبار

معرف التطبيق ومعرفات جميع أنواع الإعلانات (Banner, Interstitial, Native, Rewarded) تستخدم قيم الاختبار (.../ca-app-pub-3940256099942544). يجب إنشاء حساب AdMob فعلي وإنشاء وحدات إعلانية حقيقية واستبدال هذه القيم. الإطلاق بهذه المعرفات لن ينتج أي إيراد.

نظام الاشتراكات - غير موجود

Google Play Billing Library غير مضافة

رغم وجود دعم isPremiumUser في AdManager وAdPreferences، لا يوجد أي تكامل فعلي مع Google Play Billing Library. لا يمكن حاليا شراء اشتراك مميز أو إدارة دورة حياة الاشتراك. الفحص الحالي للحالة المميزة هو محلي فقط ولا يتصل بأي خدمة فعلية.

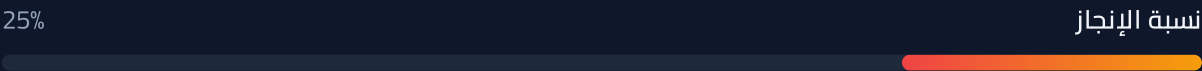
لتطبيق نموذج Freemium المقترح، يجب إضافة Google Play Billing Library (com.android.billingclient:billing) وتنفيذ منطق الشراء والاسترداد. الميزات المقترحة للنسخة المميزة تشمل: إزالة جميع الإعلانات، تصدير PDF بدون علامة مائية، عدد غير محدود من المشاريع المحفوظة، تحليلات متقدمة، ومزامنة سحابية عبر Firebase. يجب أيضا تحديد مستويات الاشتراك (شهري/سنوي) مع خصم للسنة الأولى لتشجيع الاشتراك المبكر.

العنصر	الحالة الحالية	المطلوب
AdMob App ID	معرف اختبار	إنشاء حساب AdMob واستبدال المعرفات
Banner Ads	مكونات جاهزة (معرفات اختبار)	استبدال المعرفات واختبار الإنتاج
Interstitial Ads	مدمج مع تحكم التكرار	استبدال المعرفات
Native Ads	عنصر نائب فقط (Placeholder)	تنفيذ Native Advanced Ad كامل
Rewarded Ads	معرف اختبار فقط	تنفيذ مكافآت مقابل ميزات مميزة مؤقتة
Google Play Billing	غير موجود	إضافة المكتبة وتنفيذ الشراء
التحقق من المشتريات	محلي فقط	التحقق من الخادم لحماية من التلاعب
GDPR / UMP	مدمج وجاهز	اختبار على أجهزة حقيقية في الاتحاد الأوروبي

المرحلة الخامسة

CI/CD والاختبارات

مكتمل 25% - البنية موجودة لكن بها مشاكل



تم إنشاء مسار CI/CD عبر GitHub Actions يتضمن 4 وظائف: فحص Lint، اختبارات الوحدة، بناء Debug APK، وبناء Release AAB (عند إضافة تاج إصدار). لكن المسار الحالي يحتوي على مشكلة درجة وهي عدم توافق

إصدار Java حيث يستخدم المسار JDK 17 بينما المشروع يتطلب JDK 21. كما أن تغطية الاختبارات شبه معدومة بوجود 10 اختبارات فقط لتطبيق يحتوي على 40 محرك حسابي و170 ملف مصدري.

مشكلة درجة: عدم توافق إصدار Java في CI/CD

ملف android.yml يستخدم '17: JAVA_VERSION' في حين أن المشروع يطلب JDK 21. هذا سيؤدي إلى فشل البناء في CI/CD. يجب تغيير القيمة إلى '21' فوراً. كما أن التخزين المؤقت لـ Gradle يستخدم hashFiles لملفات gradle.kts فقط دون تضمين libs.versions.toml الذي يحتوي على أرقام الإصدارات، مما قد يؤدي إلى استخدام تخزين مؤقت قديم عند تغيير الإصدارات فقط.

حالة الاختبارات الحالية

تغطية اختبارية شبه معدومة - 10 اختبارات فقط

يوجد 3 ملفات اختبار فقط: ExampleUnitTest (اختبار افتراضي 2+2=4 بخطأ في اسم الحزمة)، CalculatorEngineTest (5 اختبارات لسلامة العوارض والأسطوانات والقواعد)، وECPColumnTest (4 اختبارات لقدرة المحورية وتسليح الأعمدة). لا يوجد أي اختبار لـ ViewModels أو Repositories أو DAOs أو مكونات Compose.

تغطية الاختبارات الحالية أقل من 2 بالمائة من الكود وهي نسبة غير مقبولة لأي تطبيق إنتاجي. مع وجود 16 ViewModel و11 DAO و40 محرك حسابي، يجب إنشاء اختبارات على الأقل لكل ViewModel (اختبارات الحالة والتحويلات) وكل محرك حسابي (اختبارات القيم الحدية والنتائج المتوقعة). يُنصح باستخدام مكتبة Turbine لاختبار StateFlow MockKg للكائنات الوهمية.

العنصر	الحالة	الأولوية
إصلاح إصدار Java في CI/CD (17 إلى 21)	لم يُصلح بعد	درجة
إعداد Secrets للتكوين (keystore, passwords)	غير موجود	عالية
اختبارات (16 ViewModel ViewModels)	0 اختبارات	عالية
اختبارات محركات الحسابات (40 محرك)	9 اختبارات فقط	عالية
اختبارات DAOs (11 واجهة)	0 اختبارات	متوسطة
اختبارات UI Compose	0 اختبارات	متوسطة
تقرير تغطية الاختبارات (JaCoCo)	غير موجود	متوسطة
تخزين مؤقت يتضمن libs.versions.toml	غير مكتمل	منخفضة

خريطة الطريق والأولويات

التوصيات حسب الأولوية - الأسبوعان القادمان

الأول: إصلاح إصدار Java في CI/CD من 17 إلى 21 فوراً لأنه يمنع البناء التلقائي بالكامل. الثاني: إنشاء Room Migration Strategy لأنها حماية أساسية لبيانات المستخدم. الثالث: إضافة Firebase Crashlytics لتتبع الأعطال قبل أي إصدار إنتاجي. الرابع: بدء اختبارات الوحدة لمحركات الحسابات لأنها المنطق الأكثر أهمية في التطبيق.

بناءً على التحليل الشامل للمشروع، الوضع الحالي يمكن وصفه بأنه أساس متين يحتاج إلى تنظيم وترتيب. النقطة الإيجابية الكبرى هي أن محركات الحسابات مبنية بشكل ممتاز مع نمط الاستراتيجية ودعم 3 أكواد تصميم. لكن الهيكل المعماري المحيط بها يحتاج إلى ترتيب قبل إضافة ميزات جديدة. الهدف في المرحلة القادمة يجب أن يكون تثبيت الأساس (إصلاح CI/CD، حماية البيانات، اختبارات أساسية) بدلاً من إضافة ميزات جديدة فوق أساس غير مستقر.

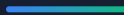
ملاحظة حول ترتيب التنفيذ

يُنصح بشدة بعدم البدء بالتنظيم النمطي (Multi-module) إلا بعد استقرار الاختبارات. فصل المشروع إلى وحدات متعددة بدون اختبارات كافية قد يُقدم مشاكل جديدة (تبعيات دائرية، وصول خاص، بناء معقد). الأفضل هو: استقرار CI/CD ثم اختبارات أساسية (30 بالمائة تغطية) ثم Firebase Crashlytics ثم إصلاح استراتيجية الترحيل ثم التنظيم النمطي.

نقاط القوة الحالية

نمط الاستراتيجية في محركات الحسابات ممتاز - Hilt DI مدمج بشكل صحيح - Room مع 11 كيانا جاهزاً - نظام ثنائي اللغة عربي/إنجليزي يعمل - 16 ViewModel جاهزة - دعم 3 أكواد تصميم (SBC/ECP/ACI) - رسومات هندسية احترافية - نظام إعلانات غير مزعج - أمان التطبيق (Play Integrity, ProGuard) - التوافق مع الأجهزة اللوحية والهواتف.

CivileG2



هذا المستند هو تحليل تقني لحالة المشروع مقارنة بخطة التطوير الخمسية. الأرقام والأولويات مبنية على فحص الكود المصدري الفعلي وليس تقديرات نظرية. يُوصى بمراجعة هذا التحليل قبل بدء أي مرحلة تطوير جديدة.

يوليو 2026 - الإصدار 1.1.0